BUILDING LLM-POWERED APPLICATIONS · LECTURE 6

Retrieval-Augmented Generation

Ground the model in your data — retrieve, then read.

Press F for fullscreen · navigate with ← →

You ask a plain chatbot (no tools, no docs): "What's on page 3 of OUR course handbook?" Most likely outcome?

- It says it cannot see your handbook and asks you to paste it
- **It invents a confident, plausible-sounding page 3 that doesn't exist ✓**
- It refuses to answer anything about handbooks
- It returns the real page 3 from its training data

Today: ground answers in YOUR data

By the end you can

- Say why RAG grounds & cites where fine-tuning doesn't
- Trace retrieve → assemble → generate → cite
- Run a mini-RAG that answers from context, not memory
- Assemble a prompt that says *"I don't know"* when the fact is absent
- Name the common retrieval failure modes

Builds on Lecture 5

You built a vector store last week. RAG is what we *do* with it: look up the right passages, then let the model read them.

This is the retrieval half of your app — and the core of HW3.

Why RAG?

An LLM only knows its training data and the messages you send. Ask about your PDFs, your wiki, last week's tickets — and it guesses, fluently and wrongly.

RAG fixes three things

- Grounding — answer from real data
- Less hallucination — facts in context
- Citations — show the source

vs. fine-tuning

No retraining. Update the knowledge base and the next answer is fresh — cheap, auditable, always current.

RAG or fine-tune?

Both teach the model new facts. They differ in where the knowledge lives — and that changes everything.

Fine-tune

Bakes facts into the weights. Expensive, slow, and *stale the moment your data changes* — edit a doc and you must retrain.

RAG

Keeps facts in an external store you update any time. Re-index a changed doc and the next answer is current. Cheaper & auditable.

For knowledge that changes — docs, policies, catalogues — RAG is almost always the right tool. The source of truth stays *outside* the model.

Retrieve-then-read

The whole pattern in one sentence: find the relevant chunks, then let the model read them to answer.

```
# 1. embed the question into a vector
q = embed(question)
# 2. search the vector store for nearest chunks
hits = store.search(q, k=4)
# 3. stuff those chunks into the prompt as context
prompt = assemble(question, hits)
# 4. generate an answer grounded in that context
answer = llm(prompt)
```

The model never sees your whole corpus — only the top-k chunks retrieval hands it.

The first demo uses simple keyword overlap for `retrieve`; later we swap in real `bge-m3` embeddings — same shape, both run in the sandbox.

In retrieve-then-read, the model is given the top-k retrieved chunks plus the question. What does it actually read to answer?

- Only those few retrieved chunks — never the whole corpus ✓
- The entire knowledge base, searched live as it generates
- Just the question; the chunks are only used to pick a citation

Pipeline


User question
Embed query
Search vector store
Top-k chunks
Assemble prompt + context
LLM
Answer + citations
Chunked docs
indexed

Indexing: from PDF to chunks

Before you can retrieve, you build the index once, offline:

1. Load — extract text from PDFs / docs
2. Chunk — split into ~200–500 token passages, with a little overlap
3. Embed — each chunk → a vector
4. Store — vectors + text + source metadata

Why chunk?

A whole PDF won't fit the context window — and finer chunks let retrieval pinpoint the right paragraph, not the right book.

You built this store in Lecture 5. RAG is what we do *with* it.

Chunker

```python
# Indexing's "chunk" step in pure Python: slide a window over the WORDS, carrying
# source + page so a citation survives into every chunk. No model -> runs anywhere.

def chunk(text, size=12, overlap=4, source="doc", page=1):
    words = text.split()
    step = size - overlap                # consecutive chunks share `overlap` words
    out = []
    for start in range(0, len(words), step):
        window = words[start:start + size]
        if not window:
            break
        out.append({"text": " ".join(window), "source": f"{source} p.{page}"})
        if start + size >= len(words):    # last window reached the end
            break
    return out
```

Getting text out of a PDF

`chunk()` needs plain text, but real sources are PDFs. Extraction is a separate step with its own libraries:

```
# run in YOUR venv, NOT the class sandbox (no pypdf there)
from pypdf import PdfReader
reader = PdfReader("handbook.pdf")
pages = [{"text": p.extract_text() or "",
          "source": f"handbook.pdf p.{n}"}
         for n, p in enumerate(reader.pages, 1)]
```

Sandbox limit: `pypdf` / `pdfplumber` aren't installed here. For demos, **paste the text** or ship a pre-extracted `.txt`. Keep the shape `{"text":…, "source":"file p.N"}` so the page number survives into the citation.

Prompt assembly & context injection

Retrieval gives you chunks; now assemble them into a prompt that pins the model to that context:

```
"Answer the question using ONLY the context below."
"If the answer isn't there, say you don't know."
""
"[1] {chunk_1.text}  (source: {chunk_1.source})"
"[2] {chunk_2.text}  (source: {chunk_2.source})"
""
"Question: {question}"
"Cite sources as [1], [2]."
```

Number the chunks so the model can cite by index. "Only the context" + "say I don't know" is what curbs hallucination.

You assemble the prompt from the top-k retrieved chunks. You bump k from 4 to 40 to "give the model more to work with." What's the most likely effect on answer quality?

- It usually gets worse — relevant facts drown in noise and get lost in the middle of a long context ✓
- It always gets better — more context can only help the model
- No change — the model only reads the first chunk regardless of k
- It gives the same answer every time, because more chunks force temperature to 0

Our toy KB has 3 chunks: [1] term code 1/2026 = semester 1... [2] students run Python in a Piston sandbox... [3] RAG grounds answers to reduce hallucination... Retrieval = keyword overlap, k=2. For the query "How do students run Python?", which chunk ranks #1?

- [2] — it shares the words 'students', 'run', 'Python' ✓
- [1] — it's listed first in the KB
- [3] — it mentions RAG, which is the topic
- All three tie, because every chunk shares 'the'

PREDICT → RUN → MODIFY

Before you run the mini-RAG

The next slide runs two questions through our toy KB. Commit to an answer for each:

- (a) `"What does the term code 1/2026 mean?"` — is it answered, with a citation?
- (b) `"What is the capital of France?"` — a fact the model *knows* but is NOT in the KB. What *should* a grounded system say?

Write both down, then run.

Mini Rag

```python
# A whole RAG pipeline in pure Python. The "vector store" is a list of dicts and
# retrieval is keyword overlap (no embeddings call). Run with Ctrl/Cmd+Enter.
import re
from openai import OpenAI

client = OpenAI()

# --- tiny in-memory knowledge base: each chunk carries its source ---
KB = [
    {"text": "The KMITL term code 1/2026 means semester 1 of the 2026 academic year.",
     "source": "handbook.pdf p.3"},
    {"text": "Students run Python in an in-browser sandbox powered by Piston.",
     "source": "syllabus.pdf p.7"},
    {"text": "RAG grounds answers in your own documents to reduce hallucination.",
     "source": "lecture-6.pdf p.1"},
]
```

The one seam in the whole pipeline

`answer()` never mentions embeddings, keywords, or scores. It only trusts a contract:

```
# retrieve: question -> list of {"text":…, "source":…} chunks, possibly EMPTY
hits = retrieve(question)          # keyword? embedding? answer() doesn't care
context = assemble(hits)           # fixed
answer  = generate(context)        # fixed
cite(answer, hits)                 # fixed
```

The empty-list case *is* the "I don't know" trigger. Nothing else changes.

§8 swaps in a `bge-m3` retriever without touching another line — same shape.

This is the exact seam you own in HW3: keep the contract, change the insides.

THINK · PAIR · SHARE

How big should a chunk be?

Chunking is a real engineering choice. With your neighbour, reason through the two failure directions:

Too large

One chunk mixes several topics → its relevance score gets diluted, and you burn context on irrelevant text.

Too small

A stray sentence with no subject → it loses the context needed to make sense or to score well.

Discuss (2 min): for a course handbook of short policies, would you split on paragraphs, headings, or a fixed 300-word window? Why do natural boundaries usually win?

Your RAG app answers a question whose facts are NOT in any retrieved chunk. What should a well-prompted system do?

- Answer confidently from its training data anyway
- **Say it doesn't know / can't find it in the provided context ✓**
- Silently lower k and re-retrieve until any chunk scores above zero
- Throw an exception so the request fails

Returning citations

A grounded answer without a source is just a nicer-looking guess. Always return where it came from.

Two layers

- In-text markers: `[1]`, `[2]`
- Source list built from the retrieved chunks' metadata

Trust, not vibes

Citations let the user *verify* the claim and let you debug bad answers back to the chunk that caused them.

Keep `source` (file + page) on every chunk at index time — you can't cite metadata you never stored.

Your RAG answer is correct AND grounded in the retrieved chunks. Why still attach a citation (file + page) to it?

- So the user can verify the claim, and so you can debug a bad answer back to its chunk ✓
- It makes the model retrieve better chunks next time
- Citations are what stop the model from hallucinating in the first place

Failure modes

Most "bad RAG" is bad retrieval, not a bad model.

Bad chunks

Wrong, partial, or off-topic passages → the model answers from garbage. Garbage in, garbage cited.

Nothing relevant

The answer isn't in the corpus, but a weak prompt makes the model invent one anyway.

Lost in the middle

Models attend best to the start and end of a long context. Facts buried in the middle get ignored — keep top-k small and ordered.

Stale index

Docs changed, embeddings didn't. Re-index when the source changes.

Right chunk retrieved — still wrong?

When `retrieve(q)` *did* return the right chunk, the bug is in the text-handling between retriever and model.

Symptom	Cause	Fix
Cites the wrong `[n]`	Numbering drifted from order	Number at assembly; parse `[n]` back
Ignores a retrieved chunk	Buried in the middle	Reorder: best chunk to an edge
Confident off-topic answer	Low score slipped through	Add a `min_score` floor
Same fact repeated 3×	Near-duplicate chunks	Dedupe before assembly
"I don't know" but it's there	Query phrased unlike chunks	Rewrite the query first

What a retrieval score means (and the `min_score` floor)

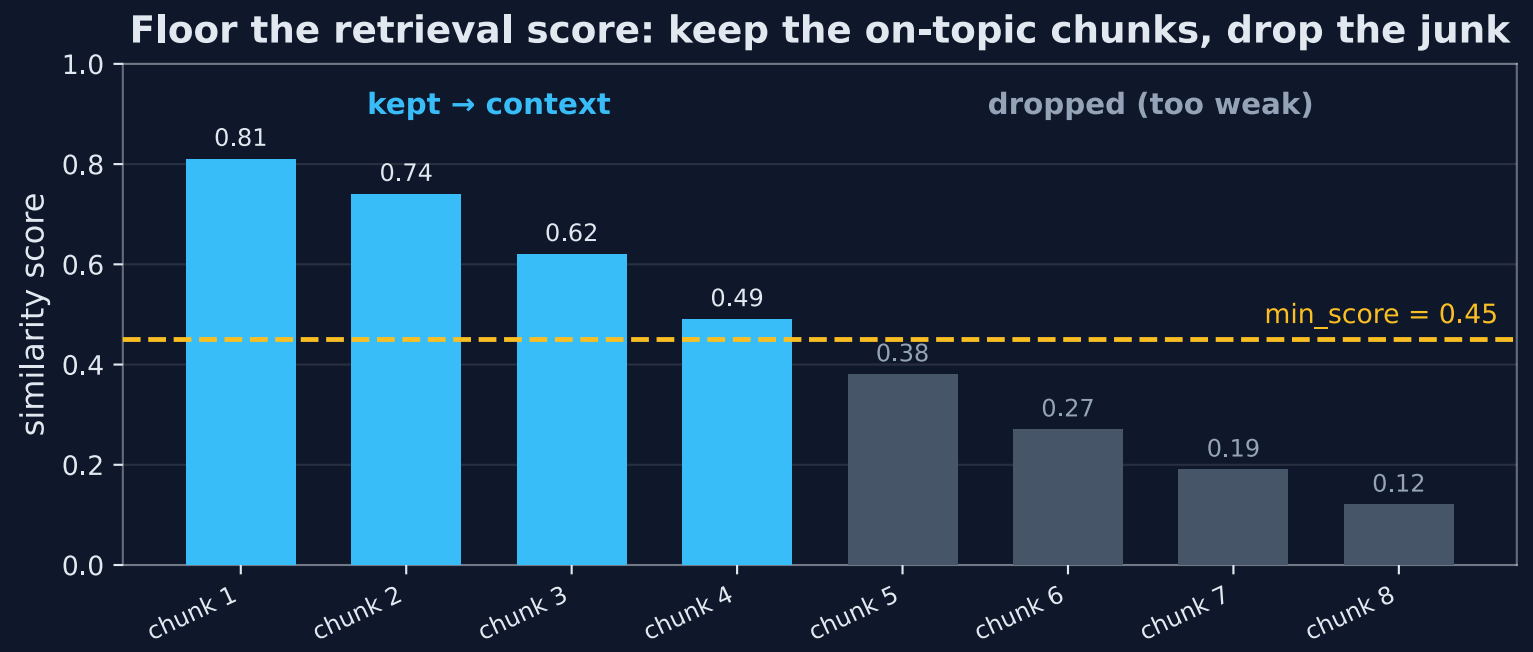
A relevance score is just a number: higher = more on-topic. Similarity scores run from `0.0` (unrelated) to `1.0` (near-identical).

Set a min_score floor — say `0.45`. Anything below it is weak; drop it.

If nothing clears the floor, answer "I don't know" instead of citing junk.

This is the scale the symptom table and the confidence poll already assume — a fixed cutoff on a 0→1 score.

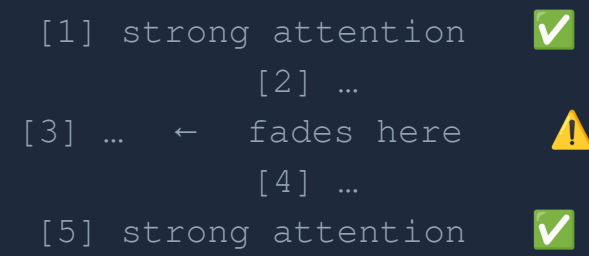
The min_score floor in action



Keep chunks above the floor as context, drop the weak matches — and if nothing clears the bar, answer "I don't know" instead of citing junk.

Lost in the middle

LLMs attend best to the start and end of a long context. A fact buried in the middle of many chunks gets quietly ignored — even when retrieval found it.



Keep k small — 3–5 good chunks beat 20 noisy ones.

Put the most relevant chunk first, not buried mid-list.

More context is not automatically better context — it's why the top-k poll punished `k=40`.

Fix the query, not the retriever

Symptom: retrieval misses a chunk that's clearly in the corpus. The retriever isn't broken — the *query* is.

Raw query

`"how do i run my code?"`

Conversational, pronoun-y, keyword-poor → lands far from the chunk vector.

Rewritten

`"run execute Python in-browser sandbox Piston"`

Expanded keywords + resolved context → lands near the chunk.

Two moves: expand (synonyms/keywords) and decontextualise (resolve "it", "that one" from chat history).
Rule: if *you* can't tell what a query is about in isolation, neither can the retriever.

Reorder against lost-in-the-middle

Symptom: 6 good chunks, answer is in #3, model acts blind. Models attend best to the start and end — so fold the ranking outward, strongest evidence to the edges.

```
def reorder(hits):           # hits sorted best-first
    head, tail = [], []
    for i, h in enumerate(hits):
        (head if i % 2 == 0 else tail).append(h)
    return head + tail[::-1]
# ['A-best','C','E-worst','D','B'] -> A & B frame the context
```

Only matters once context is *long*. Number chunks after reordering so `[1]` is what you placed first. Keeping `k` small is the better first move.

Retrieval returns top scores [0.30, 0.20, 0.18] and your min_score floor is 0.45. What should answer() do?

- Assemble the three chunks and let the model decide — it'll sort it out
- **Short-circuit: don't call the model at all, return "I couldn't find that in the documents"** ✓
- Pick the top chunk (0.30) anyway — some context beats none
- Lower min_score to 0.15 so something always comes back

Dedupe before you spend a slot

Symptom: two of your top-`k` slots say the *same* thing — chunk overlap and duplicate docs crowd out a different fact.

```
def dedupe(hits, threshold=0.8):    # best-first
    words = lambda t: set(t.lower().split())
    kept = []
    for h in hits:
        hw = words(h)
        if all(len(hw & words(k)) / len(hw) < threshold for k in kept):
            kept.append(h)
    return kept
```

Walk best-first, drop any chunk too similar to one already kept → the highest-ranked of each duplicate group survives; the freed slot goes to a genuinely new chunk. With embeddings, compare vectors (`cosine ≥ threshold`) instead of words — same loop.

Rag Trace

```
# End-to-end RAG trace, every intermediate printed:
# rewrite -> retrieve by meaning -> min_score floor -> dedupe -> assemble -> answer.
import math
from openai import OpenAI

client = OpenAI()
KB = [
    {"text": "Students run Python in an in-browser sandbox powered by Piston.", "source":
"syllabus.pdf p.7"},
    {"text": "Students execute Python in the browser; Piston runs the sandbox.", "source":
"faq.pdf p.2"},  # near-dup
    {"text": "The KMITL term code 1/2026 means semester 1 of academic year 2026.",
"source": "handbook.pdf p.3"},
]
emb = lambda ts: [d.embedding for d in client.embeddings.create(model="bge-m3",
input=ts).data]
vecs = emb([c["text"] for c in KB])
```

Two RAG answers to "When is the final exam?" The corpus has NO exam date. Which answer shows the system is working CORRECTLY?

- "The final exam is on 15 May 2026." — specific and confident
- **"I couldn't find the exam date in the provided documents." ✓**
- "The exam is probably in May, like most KMITL courses."
- "Error: no matching chunk found." — crashes the request

Your RAG answer ends "...as stated in [2]", but the claim actually appears in chunk [1] — and chunk [1] WAS retrieved (it's in the top-k). Where is the bug?

- Assembly / citation — the right chunk was retrieved, but the model cited the wrong marker (or your numbering doesn't match) ✓
- Retrieval — the wrong chunk was returned, so re-tune the retriever
- The embedding model — switch from bge-m3 to a larger one
- The context window — it's too short to hold chunk [1]

From toy to production

Our demo scored chunks by keyword overlap. Real systems use embedding similarity — but the pipeline shape is identical.

Toy (today)

- List of dicts as the store
- Keyword overlap score
- Pure Python, no deps

Real

- `bge-m3` embeddings → vectors
- Cosine nearest-neighbour
- Stored in `pgvector` (or Qdrant...)

Keyword overlap misses synonyms ("car" vs "automobile"); embeddings match on meaning. Next slide runs it — `bge-m3` works right in the sandbox. Lecture 8 adds hybrid search to get both.

Our KB chunk says "Students run Python in an in-browser sandbox powered by Piston." The query is "How do learners use the coding environment?" — same meaning, but it shares no actual words with the chunk. Retrieval only matches shared words. What does it find?

- Nothing — no shared words means no match, so it returns an empty result ✓
- The Piston chunk — word matching can tell that learners means students
- All three chunks tie, so it returns the first two
- The RAG chunk, because 'use' relates to 'documents'

Embedding Retrieval

```python
# The SAME RAG pipeline, but retrieve() now uses real bge-m3 embeddings instead
# of keyword overlap. The query shares almost NO words with the Piston chunk, yet
# it's retrieved by meaning. "I don't know" now comes from a score THRESHOLD.
import math
from openai import OpenAI

client = OpenAI()

KB = [
    {"text": "The KMITL term code 1/2026 means semester 1 of the 2026 academic year.",
     "source": "handbook.pdf p.3"},
    {"text": "Students run Python in an in-browser sandbox powered by Piston.",
     "source": "syllabus.pdf p.7"},
    {"text": "RAG grounds answers in your own documents to reduce hallucination.",
     "source": "lecture-6.pdf p.1"},
]
```

Tune Min Score

```
# Tune the min_score threshold. With bge-m3, EVERY pair scores nonzero, so the
# "I don't know" path comes from a cutoff. Print the raw scores, then pick a line.
import math
from openai import OpenAI

client = OpenAI()

KB = [
    {"text": "The KMITL term code 1/2026 means semester 1 of the 2026 academic year.",
     "source": "handbook.pdf p.3"},
    {"text": "Students run Python in an in-browser sandbox powered by Piston.",
     "source": "syllabus.pdf p.7"},
    {"text": "RAG grounds answers in your own documents to reduce hallucination.",
     "source": "lecture-6.pdf p.1"},
]

def embed(texts):
```

Toy → production: same shape

You wouldn't re-embed the whole KB on every run or scan it in Python. But the logic is identical — only the plumbing scales.

Concern	Toy (today)	Production
Store	Python list of dicts	`pgvector`, Qdrant, Pinecone
Retrieval	Keyword / cosine in Python	Nearest-neighbour index (`bge-m3`)
Scale	A few chunks	Millions of chunks

In prod you store vectors in pgvector (Postgres) and let the DB do the search — the same store you built in Lecture 5. The pipeline shape — embed, search, assemble, generate — never changes.

RAG that holds up

- Chunk thoughtfully. Coherent passages with a little overlap beat arbitrary splits.
- Keep top-k small. 3–5 good chunks beat 20 noisy ones — and dodge lost-in-the-middle.
- Constrain the prompt. "Use ONLY the context" + "say I don't know" curbs hallucination.
- Always carry metadata. File + page on every chunk, so every answer can cite.
- Inspect retrieval first. When an answer is wrong, look at the chunks before blaming the model.

> RAG is mostly a retrieval problem wearing an LLM costume.

In your HW3 Mini-RAG, an answer is confidently WRONG. Per this lecture, what should you inspect FIRST?

- The exact chunks retrieval returned for that question ✓
- Raise the temperature so the model is more creative
- Switch to a bigger LLM
- Add more 'be accurate' instructions to the system prompt

Recap & what's next

You can now

- Explain why RAG grounds and cites
- Run the retrieve-then-read pipeline
- Assemble a context prompt
- Return answers with source citations
- Name the common failure modes

Next lecture

Lecture 7

Revision 1 — consolidate the whole RAG block (prompting → structured output → backend → vector store → RAG) into one mental model before the midterm.

Read the lecture notes, then start HW3 (Mini-RAG over the course PDFs).